Lesson 11 Timed alarm clock

Table

1. Overview 2

2. Hardware Circuit connection check 2

2.1 LED module&passive buzzer module connection circuit 2

2.2 Button module connection circuit 4

2.3 DS1302 module&1602LCD module connection circuit 4

2.4 Relay module, power adapter board, water pump, power adapter connection circuit 6

3. Open the sketch 8_1_Timed_alarm_clock 6

4. Upload the code and run 8

5. Code 12

6. Troubleshooting 22

7. Any questions and suggestions are welcome 23

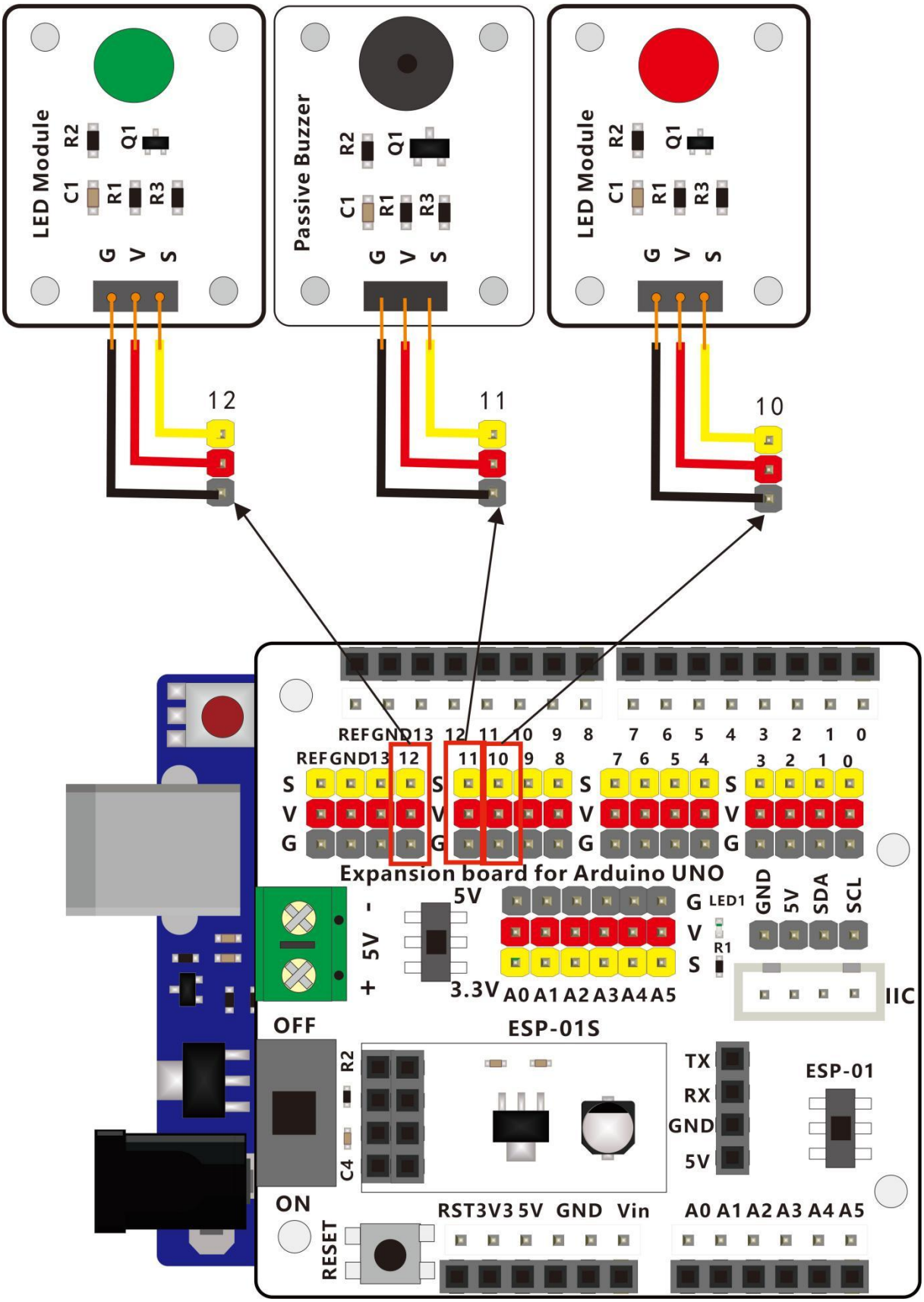
1. Overview

In the previous courses, we have understood the principles of all modules and tested their functionality using code. After successfully assembling the device according to the steps in Lesson 10, we can upload the "8_1_Timed_alarmclock" code to Arduino UNO and use these modules comprehensively to achieve the function of a timed alarm clock. The first row of the 1602 LCD displays real-time time, and the second row displays the time to set the alarm, which can be accurately set to seconds. Set the alarm time (hour: minute: second) using a combination of buttons.

2. Hardware Circuit connection check

Before uploading the code, please double check the wiring connections between all modules and the Arduino UNO expansion board according to the hardware circuit diagram. It is particularly important to note that the G and V pins of all modules are correctly connected to the G and V pins on the Arduino UNO expansion board, and must not appear (the G pin of the module is connected to the V pin on the Arduino UNO expansion board, or the V pin of the module is connected to the G pin on the Arduino UNO expansion board). If the G pin of the module is connected to the V pin on the Arduino UNO expansion board, or if the V pin of the module is connected to the G pin on the Arduino UNO expansion board, it may cause a short circuit and damage the module, Arduino UNO expansion board, and components on the Arduino UNO after power on.

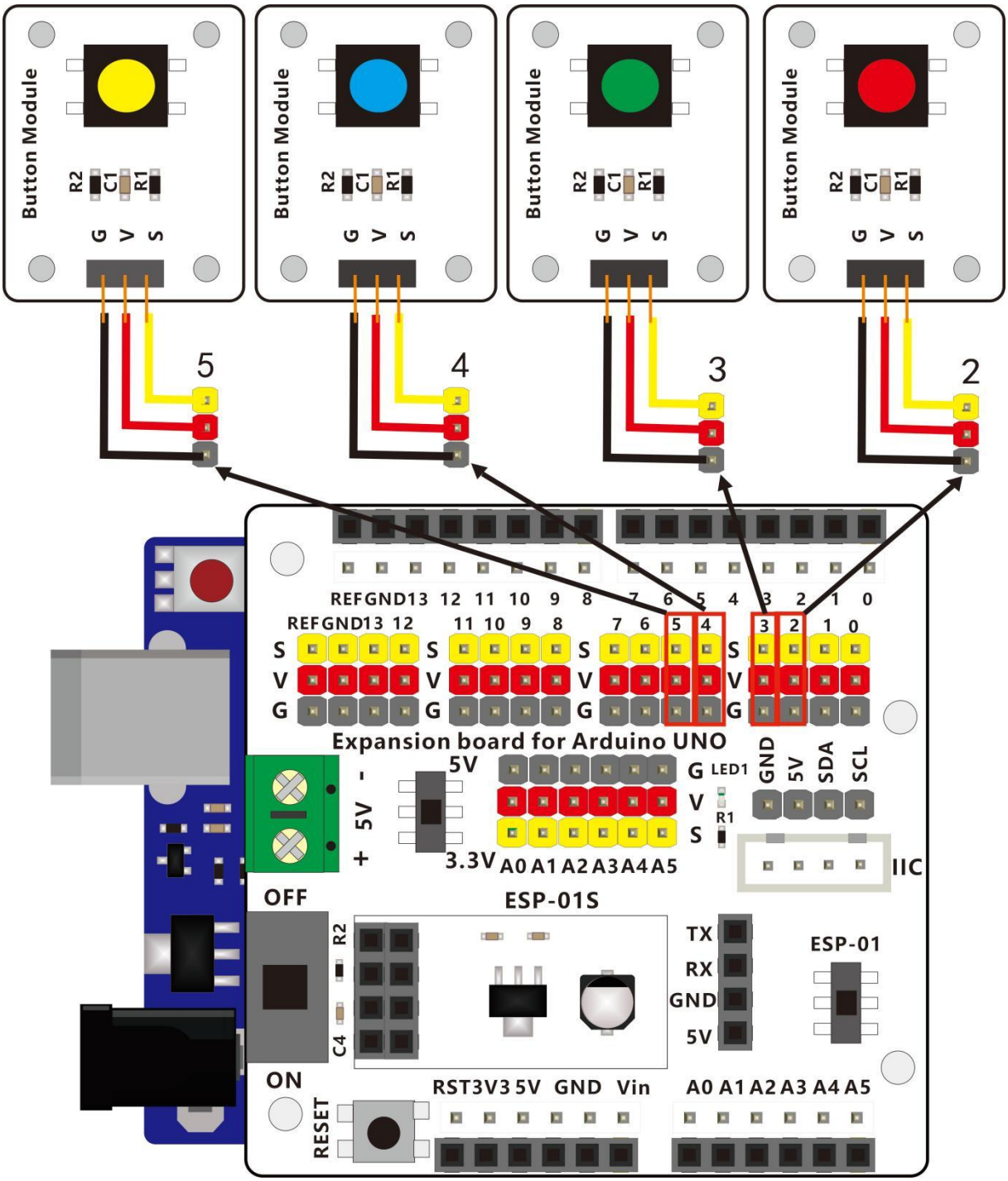
2.1 LED module&passive buzzer module connection circuit



Support email:cokoino@outlook.com

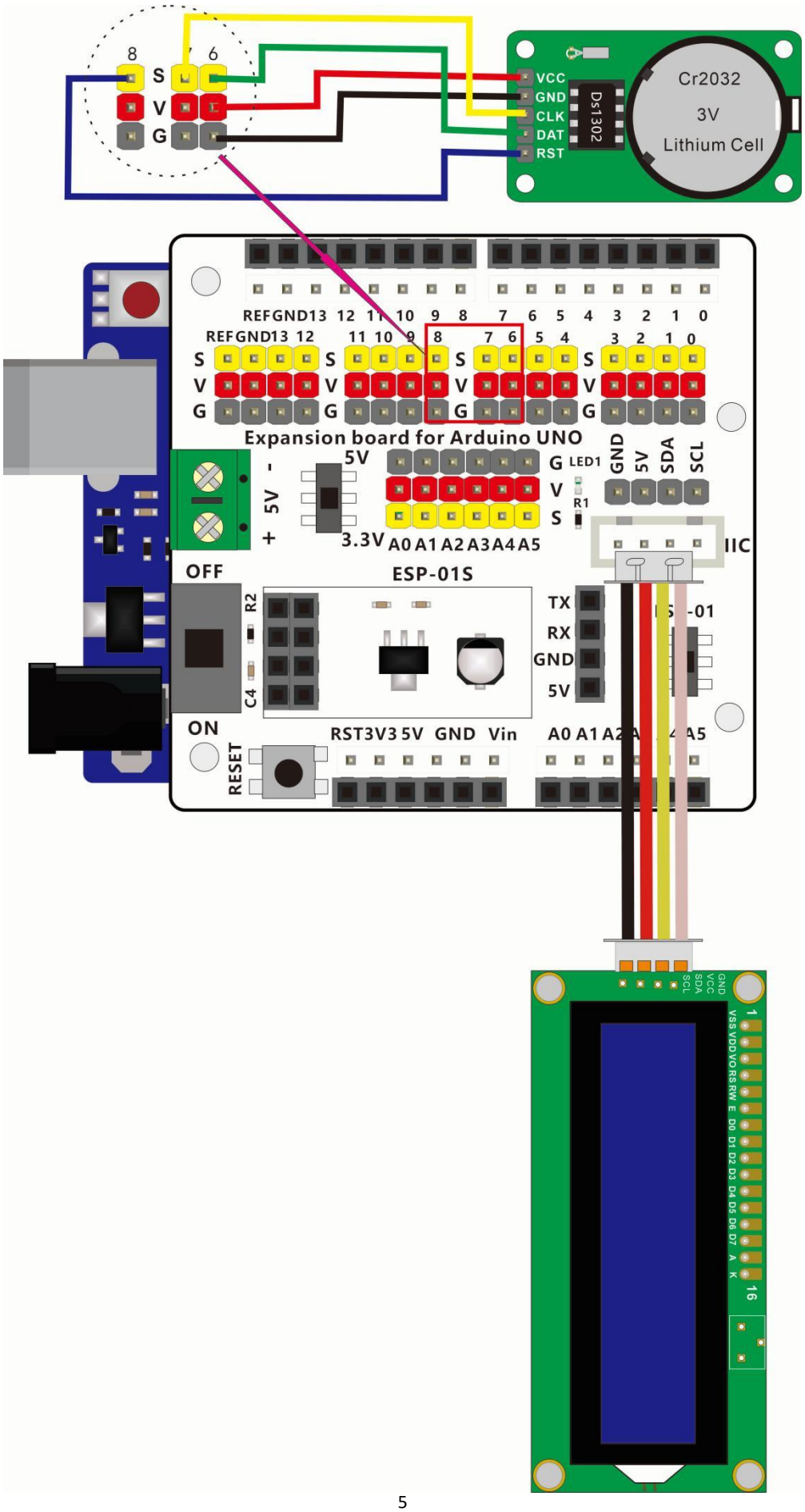
2.2 Button module connection circuit

Please refer to the following connection diagram to check if the 4 button modules are correctly connected to the Arduino UNO expansion board.



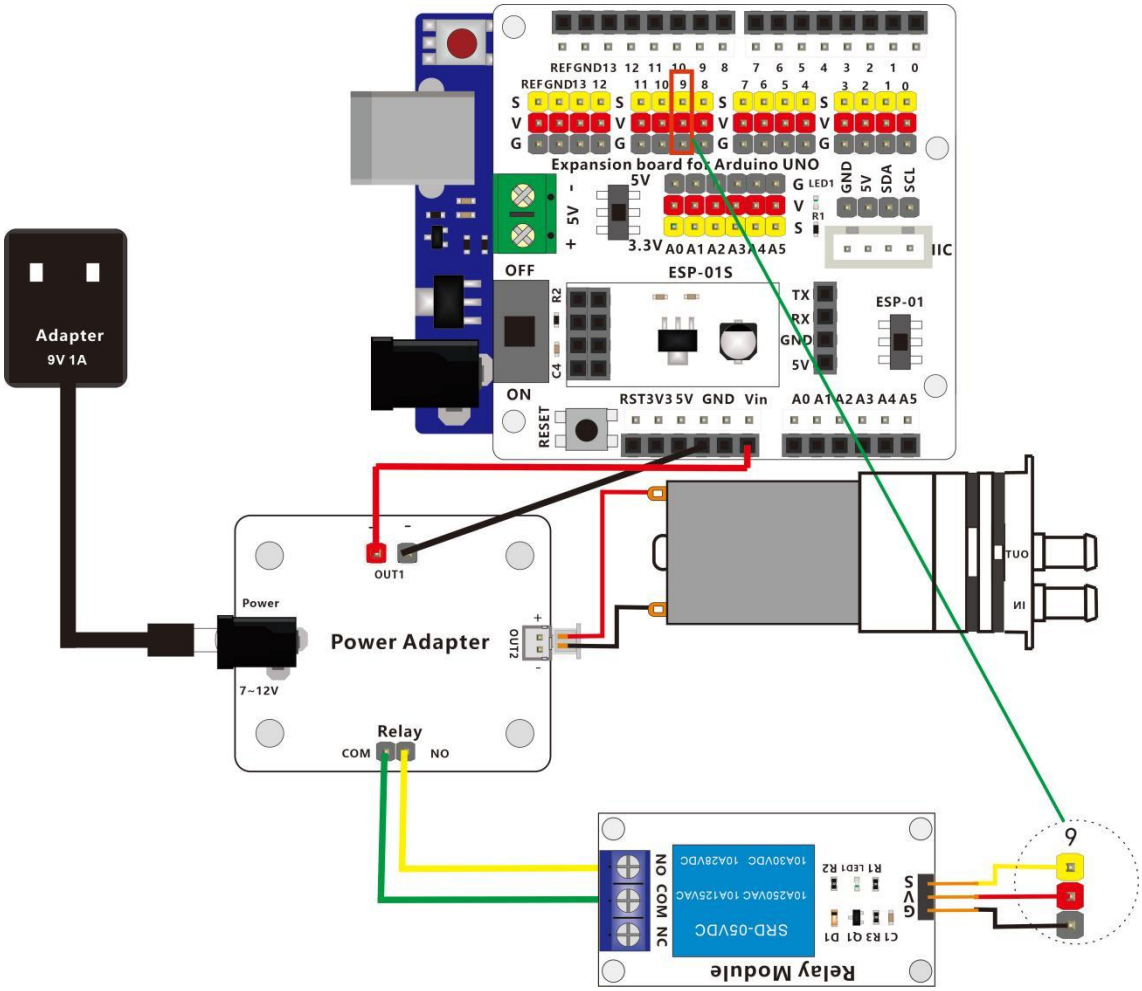
2.3 DS1302 module&1602LCD module connection circuit

Please refer to the following connection diagram to check if the DS1302 module and the 1602LCD module are correctly connected to the Arduino UNO expansion board.


8
7
6
S
V
G
VCC
GND
CLK
DAT
RST
Ds1302
Cr2032
3V
Lithium Cell
REF GND 13 12 11 10 9 8 7 6 5 4 3 2 1 0
Expansion board for Arduino UNO
5V
3.3V
A0 A1 A2 A3 A4 A5
LED1
R1
GND
5V
SDA
SCL
IIC
OFF
ON
ESP-01S
R2
C4
TX
RX
GND
5V
RESET
RST 3V3 5V GND Vin
A0 A1 A2
A5
GND
VCC
SDA
SCL

2.4 Relay module, power adapter board, water pump, power adapter connection circuit

Please refer to the following connection diagram to check if the interconnection between the relay module, power adapter board, water pump, power adapter, and Arduino UNO expansion board are correct.

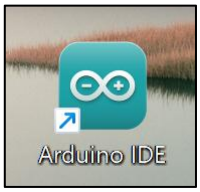


3. Open the sketch 8_1_Timed_alarm_clock

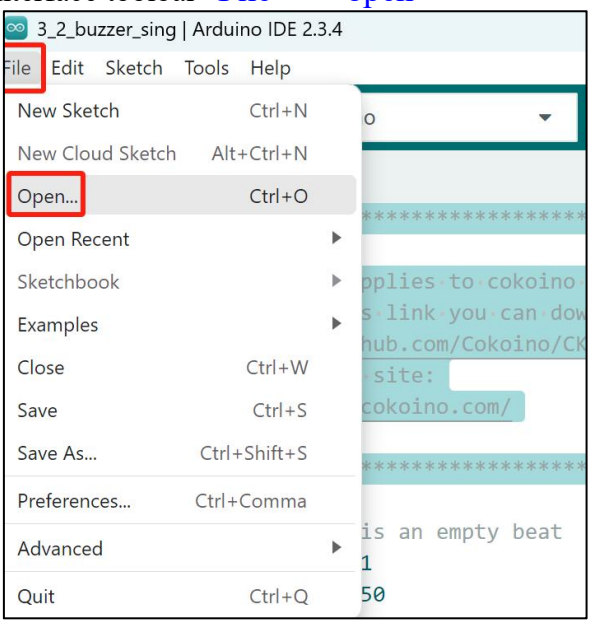
The code used in this lesson is placed in this folder：“ E:\CKK0021-main\Tutorial\sketches”

8_1_Timed_alarm_clock

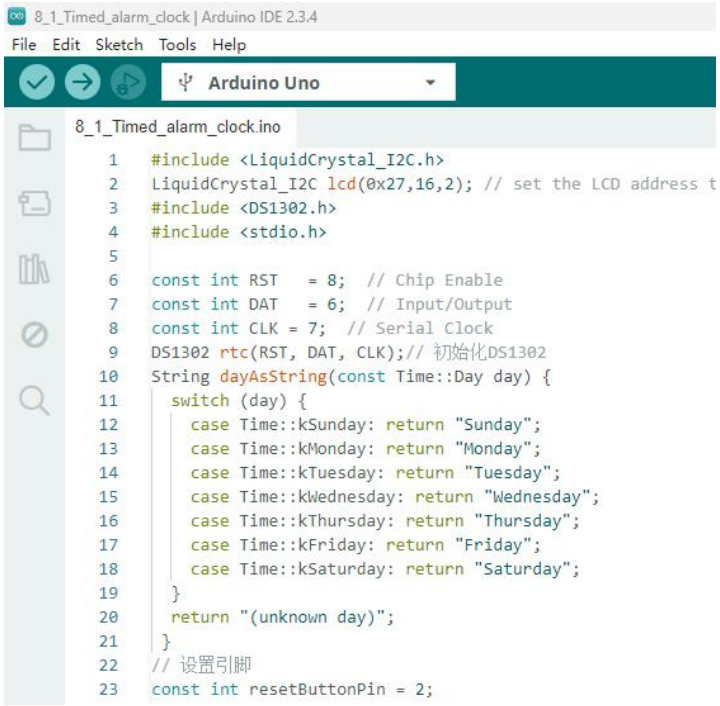
Double-click the Arduino IDE shortcut on the desktop to open it



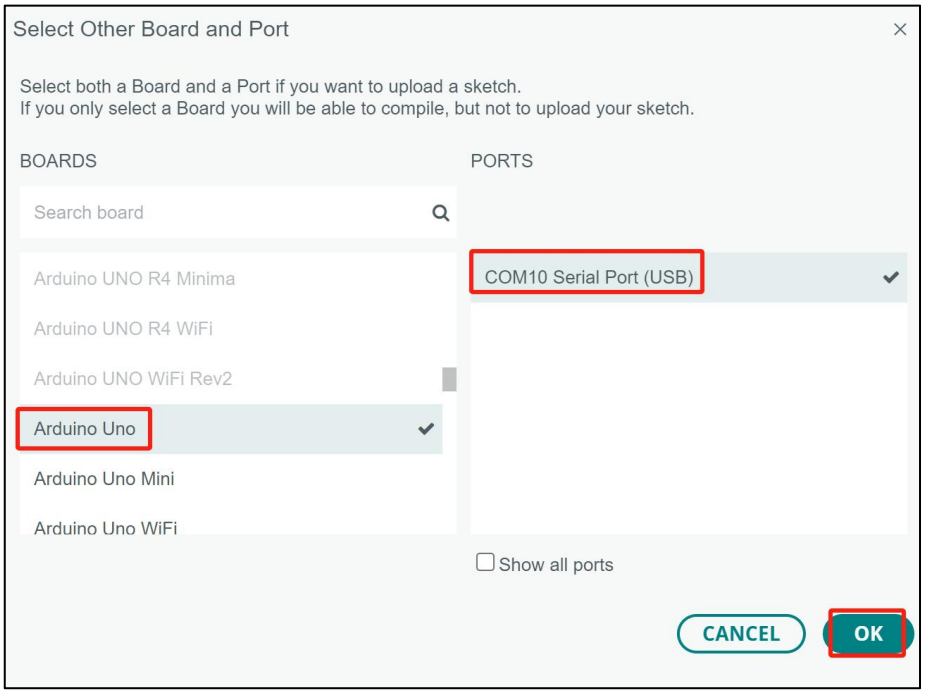
Click on the Arduino IDE interface toolbar "File" --- "open"



Select the code in the folder named 8_1_Timed_alarm_clock：
E:\CKK0021-main\Tutorial\sketches\8_1_Timed_alarm_clock,Then click "open".
The code interface after opening is as follows

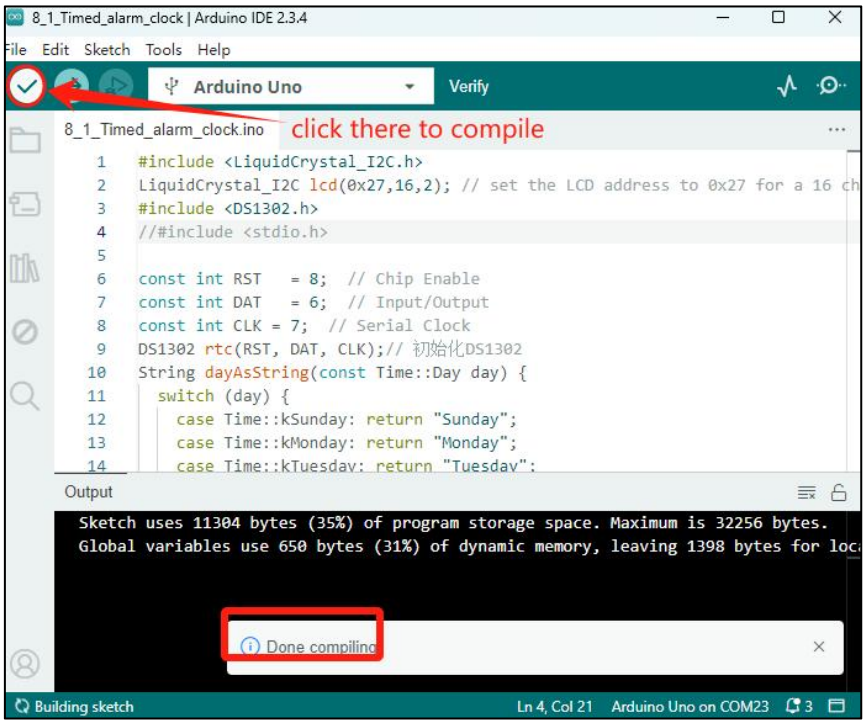


Select the board "Arduino UNO"and Port "COM10" (COM port is commonly known as an input output port for a device normally PC which enables communication between Arduino and PC. You can check your arduino com number in device manager, the com port of our arduino board is recognized as COM10 in this tutorial)

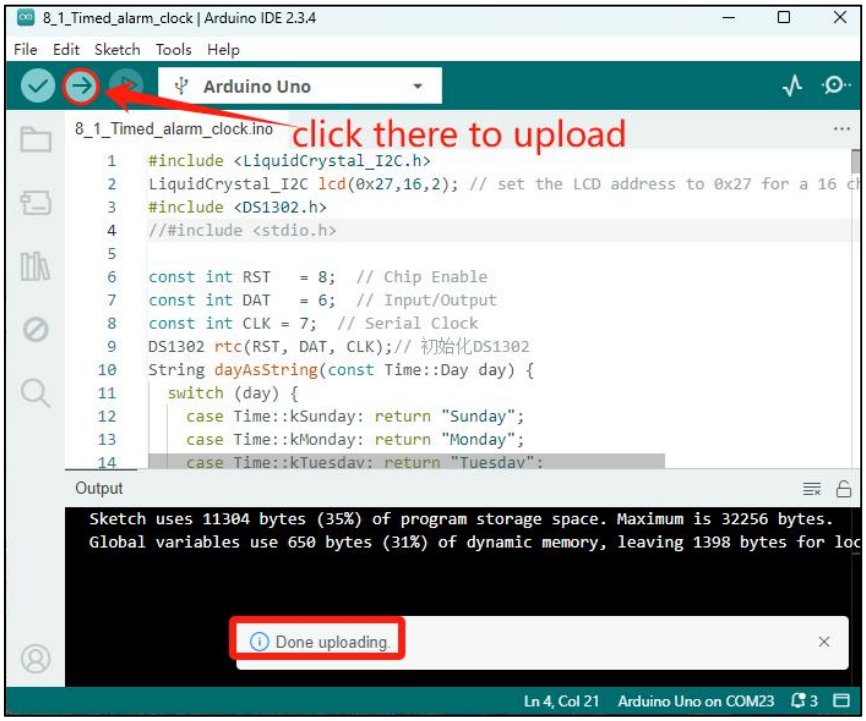


4. Upload the code and run

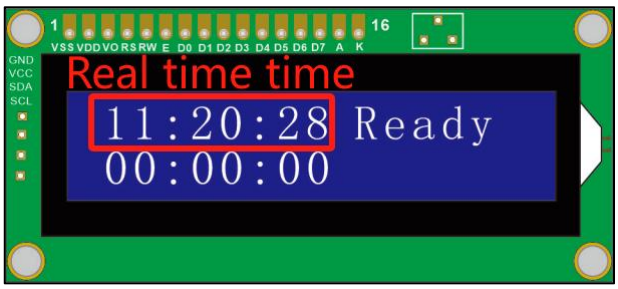
Click compile button, successfully compiled the code will display "Done compiling"



Click upload button, successfully uploading the code will display "Done uploading". When code is uploaded successfully, the program starts to run.

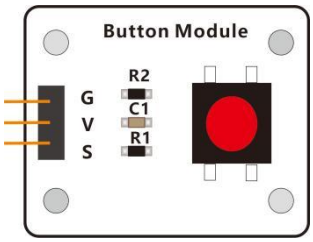


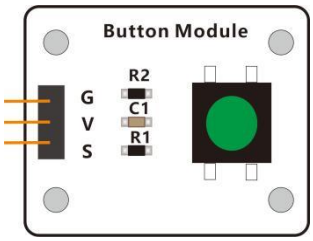
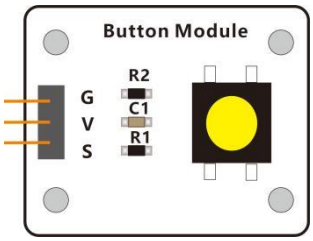
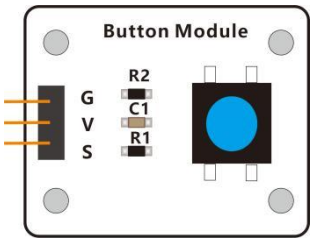
At this point, you can see the green LED light on, and the first line of the 1602 LCD screen displays the real-time time and the "Ready" string. The second line shows the time when the alarm was set, which defaults to "00:00:00".



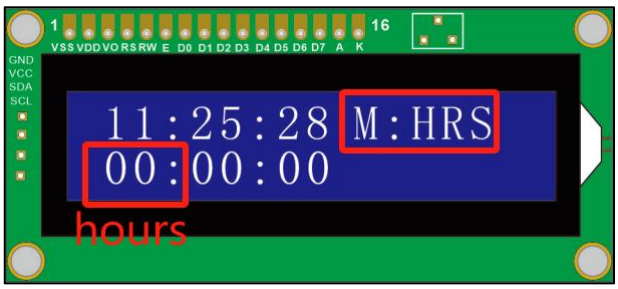
Next, let's learn how to set an alarm time through the button module. Suppose we set an alarm time of 14:50.

Please refer to the following table for the functional definition of the button module:

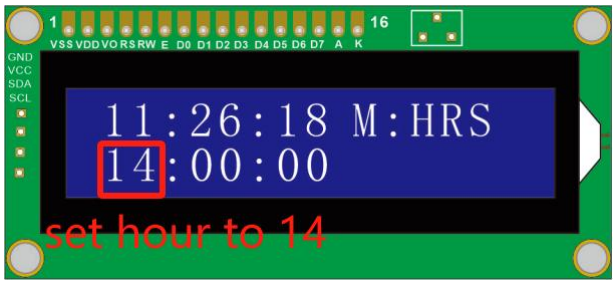
Button Module	Picture	Name	Function
Red button		Reset button	1: Press and hold for 5 seconds to enter the settings mode 2: In the setting mode, press once to save the set time data and enter idle mode

			3: In operating mode, press once to enter idle mode
Green button		Start/Stop button	1: In idle mode, press once to enter run mode (start) 2: In running mode, press once to enter idle mode (stop) 3: In setup mode, press this button to switch between hours, minutes, and seconds (switch)
Yellow button		Up button	In the setting mode, it is used to increase the values of (hour, minute, and second) The adjustable numerical range is hours (00-23), minutes (00-59), seconds (00-59).
Blue button		Down button	In the setting mode, it is used to set the values of (hour, minute, and second) to decrease downwards Adjustable numerical range: hours (23-00), minutes (59-00), seconds (59-00)

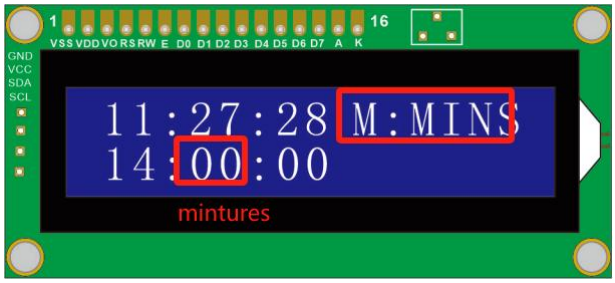
Firstly, we long press the Red (Reset) button for 5 seconds to enter the alarm setting mode, which defaults to setting the hour, as shown in the following figure:



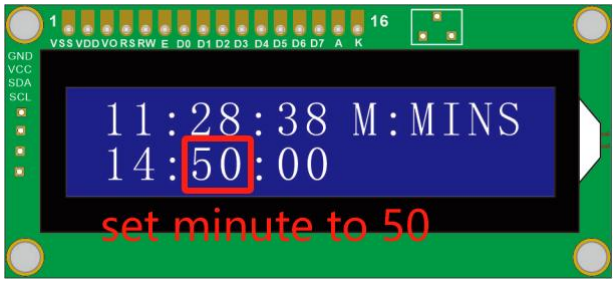
Then, we press the yellow button 14 times, and the number in the second row of the 1602 LCD displaying the hour parameter will increase from 00 to 14.



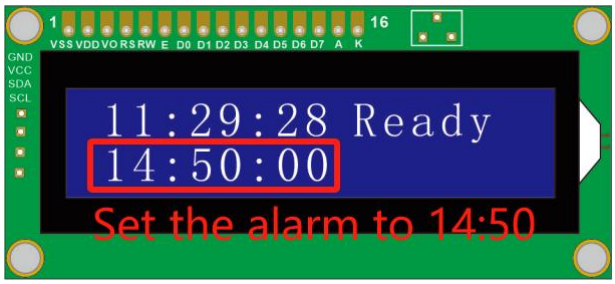
Then, we press the green button to change the setting mode from set hours to set minutes, as shown in the following figure.



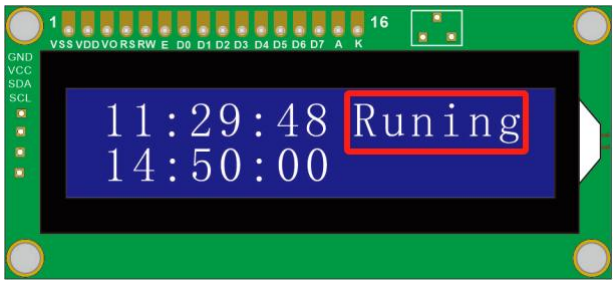
Press the blue button 10 times, and the number for the hour parameter displayed on the second row of the 1602 LCD will change from 00 to 59, and then decrease from 59 to 50.



14:50 This alarm parameter has been set. Let's press the red button once to save the setting data and enter idle mode.



Finally, we press the green button to enter the alarm running mode.



The system compares the real-time time with the set alarm time. When the real-time time reaches the set alarm time, the buzzer starts working and you can hear a "beep" sound. The green light goes out and the red light lights up. The "Buzzing" string will be displayed at the end of the first line of the 1602 LCD.

After one minute, the buzzer stops working, the green light is on, and the red light is off. Continuing with the next round of alarm clock operation mode, without changing the alarm clock parameter settings, the alarm clock will start on time again at 14:50 the next day. Of course, this is just a demonstration, you can set the alarm time according to your own needs.

5. Code

8_1_Timed_alarm_clock.ino

```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd(0x27,16,2); // set the LCD address to 0x27 for a 16 chars and 2 line display
#include <DS1302.h>
//#include <stdio.h>
const int RST     = 8;   // Chip Enable
const int DAT     = 6;   // Input/Output
const int CLK = 7;   // Serial Clock
DS1302 rtc(RST, DAT, CLK);// initialize DS1302
String dayAsString(const Time::Day day) {
  switch (day) {
    case Time::kSunday: return "Sunday";
    case Time::kMonday: return "Monday";
    case Time::kTuesday: return "Tuesday";
    case Time::kWednesday: return "Wednesday";
    case Time::kThursday: return "Thursday";
    case Time::kFriday: return "Friday";
    case Time::kSaturday: return "Saturday";
  }
  return "(unknown day)";
 }
// set up pin
const int resetButtonPin = 2;
const int startStopButtonPin = 3;
const int downButtonPin = 4;
const int upButtonPin = 5;
const int buzzerpin = 9;
const int relayPin = 10;
const int LED_R = 11;
const int LED_G = 12;
```

```
// variable
int setMode = 0; //0:set hours, 1:set minutes, 2:set seconds
int setupHours = 0;
int setupMinutes = 0;
int setupSeconds = 0;
int currentHours = 0;
int currentMinutes = 0;
int currentSeconds = 0;
bool watering = false;
bool flag = false;
int resetButtonState = LOW;
long resetButtonLongPressCounter = 0;
int startStopButtonState = LOW;
int upButtonState = LOW;
int downButtonState = LOW;
int resetButtonPrevState = LOW;
int startStopButtonPrevState = LOW;
int upButtonPrevState = LOW;
int downButtonPrevState = LOW;
bool resetButtonPressed = false;
bool resetButtonLongPressed = false;
bool startStopButtonPressed = false;
bool upButtonPressed = false;
bool downButtonPressed = false;
const int MODE_IDLE = 0;
const int MODE_SETUP = 1;
const int MODE_RUNNING = 2;
const int MODE_BUZZING = 3;
int currentMode = MODE_IDLE;      // 0=idle 1=setup 2=running 3=buzzing
                                  // Power up --> idle
                                  // Reset --> idle
                                  //   Start/Stop --> start or stop counter
                                  //   Up / Down --> NOP
                                  // Reset (long press) --> enter setup
                                  //     Start/Stop --> data select
                                  //     Up --> increase current data value
                                  //     Down --> decrease current data value
                                  //     Reset --> exit setup (idle)
int dataSelection = 0;   // Currently selected data for edit (setup mode, changes with Start/Stop)
                         // 0=hours (00-23) 1=minutes (00-59) 2=seconds (00-59)
void setup() {
  Serial.begin(9600);
```

```
  // initialize the lcd
  lcd.init();
  lcd.backlight();
  lcd.clear();
  // initialize the DS1302
  rtc.halt(false);
  rtc.writeProtect(false);
  // Set the button as input
  pinMode(resetButtonPin, INPUT_PULLUP);
  pinMode(startStopButtonPin, INPUT_PULLUP);
  pinMode(upButtonPin, INPUT_PULLUP);
  pinMode(downButtonPin, INPUT_PULLUP);
  //Set relay and LED as outputs
  pinMode(relayPin, OUTPUT);
  pinMode(LED_R, OUTPUT);
  pinMode(LED_G, OUTPUT);
  //Set the green LED to turn on
  digitalWrite(LED_G,HIGH);
}
void printTime() {
  // Get the current time and date from the chip.
  Time t = rtc.time();
  // Name the day of the week.
  const String day = dayAsString(t.day);
  // Format the time and date and insert into the temporary buffer.
  char buf[50];
  snprintf(buf, sizeof(buf), "%s %04d-%02d-%02d %02d:%02d:%02d",
           day.c_str(),
           t.yr, t.mon, t.date,
           t.hr, t.min, t.sec);
  // Print the formatted string to serial so we can see the time.
  Serial.println(buf);
  lcd.setCursor(0, 0);//Display starting from the first row and first grid of 1602 LCD
  //Display real-time time
  if((t.hr) < 10) lcd.print("0");
  lcd.print(t.hr);
  lcd.print(":");
  if((t.min) < 10) lcd.print("0");
  lcd.print(t.min);
  lcd.print(":");
  if((t.sec) < 10) lcd.print("0");
  lcd.print(t.sec);
```

```
}
void alarm_clock(){
  Time t = rtc.time();// Read the current time
  startStopButtonPressed = false;
  upButtonPressed = false;
  downButtonPressed = false;
  resetButtonPressed = false;
  resetButtonLongPressed = false;
  resetButtonState = digitalRead(resetButtonPin);
  delay(20);
  if(resetButtonState != resetButtonPrevState)
  {
    resetButtonPressed = resetButtonState == HIGH;
    resetButtonPrevState = resetButtonState;
  }
  else   // Long press management...
  {
    if(resetButtonState == HIGH)
    {
      resetButtonLongPressCounter++;
      if(resetButtonLongPressCounter == 50)//Press and hold the reset button for 5 seconds
      {
        resetButtonPressed = false;
        resetButtonLongPressed = true;
        resetButtonLongPressCounter = 0;
      }
    }
    else
    {
      resetButtonLongPressCounter = 0;
      resetButtonPressed = false;
      resetButtonLongPressed = false;
    }
  }
  /*
   * Start/Stop button management
   */
  startStopButtonPressed = false;
  startStopButtonState = digitalRead(startStopButtonPin);
  delay(20);
  if(startStopButtonState != startStopButtonPrevState)
  {
```

```
  startStopButtonPressed = startStopButtonState == HIGH;
  startStopButtonPrevState = startStopButtonState;
}
/*
 * Down button management
 */
downButtonPressed = false;
downButtonState = digitalRead(downButtonPin);
delay(20);
if(downButtonState != downButtonPrevState)
{
  downButtonPressed = downButtonState == HIGH;
  downButtonPrevState = downButtonState;
}
/*
 * Up button management
 */
upButtonPressed = false;
upButtonState = digitalRead(upButtonPin);
delay(20);
if(upButtonState != upButtonPrevState)
{
  upButtonPressed = upButtonState == HIGH;
  upButtonPrevState = upButtonState;
}
/*
 * Mode management
 */
switch(currentMode)
{
  case MODE_IDLE:
    if(resetButtonPressed)
    {
      Reset();
    }
    if(resetButtonLongPressed)
    {
      currentMode = MODE_SETUP;
    }
    Serial.println(flag);
    if(startStopButtonPressed || flag)
    {
```

```
      currentMode = currentMode == MODE_IDLE ? MODE_RUNNING : MODE_IDLE;
      flag = false;
    }
    break;

  case MODE_SETUP:
    if(resetButtonPressed)
    {
      currentHours = setupHours;
      currentMinutes = setupMinutes;
      currentSeconds = setupSeconds;
      dataSelection = 0;
      currentMode = MODE_IDLE;
    }
    if(startStopButtonPressed)
    {
      // Select next data to adjust
      dataSelection++;
      if(dataSelection == 3)
      {
        dataSelection = 0;
      }
    }
    if(downButtonPressed)
    {
      switch(dataSelection)
      {
        case 0: // hours
          setupHours--;
          if(setupHours == -1)
          {
            setupHours = 23;
          }
          break;
        case 1: // minutes
          setupMinutes--;
          if(setupMinutes == -1)
          {
            setupMinutes = 59;
          }
          break;
        case 2: // seconds
```

```
            setupSeconds--;
            if(setupSeconds == -1)
            {
              setupSeconds = 59;
            }
            break;
        }
      }
      if(upButtonPressed)
      {
        switch(dataSelection)
        {
          case 0: // hours
            setupHours++;
            if(setupHours == 24)
            {
              setupHours = 0;
            }
            break;
          case 1: // minutes
            setupMinutes++;
            if(setupMinutes == 60)
            {
              setupMinutes = 0;
            }
            break;
          case 2: // seconds
            setupSeconds++;
            if(setupSeconds == 60)
            {
              setupSeconds = 0;
            }
            break;
        }
      }
      break;
    case MODE_RUNNING:
      if(startStopButtonPressed)
      {
        currentMode = MODE_IDLE;
      }
      if(resetButtonPressed)
```

```
      {
        Reset();
        currentMode = MODE_IDLE;
      }
      break;
    case MODE_BUZZING:
      if(resetButtonPressed || startStopButtonPressed || downButtonPressed || upButtonPressed)
      {
        currentMode = MODE_IDLE;
      }
      break;
  }
  /*
   * Time management
   */
  switch(currentMode)
  {
    case MODE_IDLE:
    case MODE_SETUP:
      // NOP
      break;
    case MODE_RUNNING:
      if(t.hr == setupHours && t.min == setupMinutes)//Set alarm time, hours and minutes
       {
        currentMode = MODE_BUZZING;//Enter buzzer working mode
         }
      break;
    case MODE_BUZZING:
      digitalWrite(LED_R,HIGH);//The red LED is on
      digitalWrite(LED_G,LOW);//The green LED is off
      //The buzzer sounds for one minute, and you can modify the parameters yourself to change the duration of the
buzzer sound
      for(int i = 0;i < 30; i++)
      {
        tone(buzzerpin, 262);   //output the sound with frequency of 262Hz
        delay(250);     //delay in 250ms
        tone(buzzerpin, 294);;     //output the sound with frequency of 294Hz
        delay(250);     //delay in 250ms
        tone(buzzerpin, 330);
        delay(250);
        tone(buzzerpin, 349);
        delay(250);
```

```
        tone(buzzerpin, 392);
        delay(250);
        tone(buzzerpin, 440);
        delay(250);
        tone(buzzerpin, 494);
        delay(250);
        tone(buzzerpin, 532);
        delay(250);
        noTone(buzzerpin);     //stop sound output
      }
      digitalWrite(LED_R,LOW);
      digitalWrite(LED_G,HIGH);
      currentMode = MODE_IDLE;
      break;
}
/*
 * LCD management
 */
//lcd.clear();
lcd.setCursor(9, 0);
switch(currentMode)
{
  case MODE_IDLE:
    lcd.print("Ready   ");
    break;
  case MODE_SETUP:
    lcd.print("M:");
    switch(dataSelection)
    {
      case 0:
        lcd.print("HRS ");
        break;
      case 1:
        lcd.print("MINS");
        break;
      case 2:
        lcd.print("SECS");
        break;
    }
    break;
  case MODE_RUNNING:
    lcd.print("Runing ");
```

```
        break;
      case MODE_BUZZING:
        lcd.print("BUZZING");
        flag = true;
        break;
    }
lcd.setCursor(0, 1);
  switch(currentMode)
  {
    case MODE_IDLE:
      if(currentHours < 10) lcd.print("0");
      lcd.print(currentHours);
      lcd.print(":");
      if(currentMinutes < 10) lcd.print("0");
      lcd.print(currentMinutes);
      lcd.print(":");
      if(currentSeconds < 10) lcd.print("0");
      lcd.print(currentSeconds);
      break;
    case MODE_SETUP:
      lcd.setCursor(0, 1);
      if(setupHours < 10) lcd.print("0");
      lcd.print(setupHours);
      lcd.print(":");
      if(setupMinutes < 10) lcd.print("0");
      lcd.print(setupMinutes);
      lcd.print(":");
      if(setupSeconds < 10) lcd.print("0");
      lcd.print(setupSeconds);
      break;
  }
}

void Reset()
{
  currentMode = MODE_IDLE;
  currentHours = setupHours;
  currentMinutes = setupMinutes;
  currentSeconds = setupSeconds;
}

void loop()
```

```
{
printTime();
alarm_clock();
}
```

6. Troubleshooting

A. Code compilation failed

Please check whether the required libraries for the code have been installed in the Arduino IDE. For the library installation method, please refer to Item 5 of Lesson 6, as shown in the figure below:

5. Install the LiquidCrystal Library 8

5.1 Why Use Libraries? 8

5.2 How to use a Library in the sketch 8

5.3 Install the library to the Arduino IDE 9

Method 1: 9

Method 2: 10

Method 3: 11

B. The time displayed in the real-time area on the first line at the top-left corner of the 1602LCD does not match the actual time of your computer.

Please follow Item 6 of Lesson9 to re-upload the code "7_2_get_and_set_real-time_time" to the Arduino UNO board. After confirming that the time displayed in the real- time time area on the 1602LCD is correct, re-upload the code you need to the Arduino UNO board.

C. The time setting function cannot be accessed using the buttons, or the time cannot be adjusted.

Please check whether the connection between the button modules and the UNO expansion board is correct and secure without looseness.

7. Any questions and suggestions are welcome

THANK YOU for participating in this Lesson 11 experience!

If you find any errors, omissions or you have suggestions and/or questions about this lesson, please feel free to contact us:

cokoino@outlook.com

We will make every effort to make changes and correct errors as soon as feasibly possible and publish a revised version.

If you want to learn more about Arduino, Raspberry Pi, Smart Cars, Robotics and other interesting products in science and technology, please continue to visit our Amazon Store by search for "**LK COKOINO**" on Amazon. Or visit our official website: **www.cokoino.com**

We will continue to launch fun, cost-effective, innovative and exciting products.

Thank you again for choosing Cokoino products.

LK COKOINO